STUDY GUIDE · REACT & FULL STACK · PRINT EDITION

JavaScript to Use

The Full-Stack Developer's Guide

Topics: `import/export` · `Date` · `Math` · `setInterval()` · Ternary operator · Object destructuring

Every section follows the same order: **Concept → Core syntax → Must-know methods → Pitfalls → Real-life examples → React / Node connection → Interview questions → Practice exercises.** Section 7 is a capstone project (a live React clock) that uses all six topics, and section 8 is a one-page cheat sheet.

HOW TO USE THIS GUIDE

Type every code block into your editor (a Vite React project or a plain `.js` file run with `node`) instead of only reading it. Change values, break things, and predict the output *before* you run it — the way you explore a feature as a QE. Tick the ☐ boxes in the exercises as you finish them, and use the ruled *My notes* space at the end of each section.

PROGRESS TRACKER

Topic	Read	Exercises	Revised	Interview ready
1. import / export	☐	☐	☐	☐
2. Date	☐	☐	☐	☐
3. Math	☐	☐	☐	☐
4. setInterval()	☐	☐	☐	☐
5. Ternary operator	☐	☐	☐	☐
6. Object destructuring	☐	☐	☐	☐
7. Capstone: Live Clock	☐	☐	☐	☐
8. Cheat sheet	☐	☐	☐	☐

Name: _____ Started on: ____ / ____ / _____

Contents

1. import / export (ES Modules) 5

1.1 The concept 5
1.2 Named exports 5
1.3 Default export 6
1.4 Mixing default and named 6
1.5 Renaming (aliasing) with as 6
1.6 Namespace import (* as) 6
1.7 Re-exports and “barrel” files 7
1.8 Side-effect imports 7
1.9 Dynamic import() — loading code on demand 7
1.10 How modules behave (the deep part interviewers love) 8
1.11 ES Modules vs CommonJS (full-stack essential) 8
1.12 import.meta in Vite 9
1.13 Pitfalls 9
1.14 Interview questions 9
1.15 Practice exercises 10

2. Date 11

2.1 The concept 11
2.2 Creating dates 11
2.3 The famous traps 11
2.4 Getters (reading parts) 12
2.5 Setters (changing parts) 12
2.6 Formatting for humans 12
2.7 Date arithmetic 13
2.8 Real-life examples 14
2.9 Full-stack golden rules 14
2.10 React connection 14
2.11 Interview questions 15
2.12 Practice exercises 16

3. Math 17

3.1 The concept 17
3.2 Rounding family — the most-asked section 17

3.3 Min, max, absolute, power . . . 18
3.4 Constants . . . 18
3.5 Randomness . . . 18
3.6 Trigonometry (just enough for UI work) . . . 19
3.7 Real-life examples . . . 19
3.8 Interview questions . . . 19
3.9 Practice exercises . . . 20

4. setInterval() . . . 21
4.1 The concept . . . 21
4.2 Must-know facts . . . 21
4.3 React patterns . . . 22
4.4 Real-life examples . . . 22
4.5 Interview questions . . . 22
4.6 Practice exercises . . . 23

5. The Ternary Operator . . . 24
5.1 The concept . . . 24
5.2 Truthy / falsy refresher . . . 24
5.3 Real-life examples . . . 24
5.4 Ternaries in React (the main use) . . . 25
5.5 Ternary vs && vs ?? vs || vs ?. . . . 25
5.6 Pitfalls and style . . . 25
5.7 Interview questions . . . 26
5.8 Practice exercises . . . 26

6. Object Destructuring . . . 27
6.1 The concept . . . 27
6.2 Every feature, one by one . . . 27
6.3 Destructuring in function parameters (the React superpower) . . . 28
6.4 Assigning to existing variables . . . 28
6.5 Array destructuring (you use it daily in React) . . . 28
6.6 Destructuring in loops . . . 29
6.7 Real-life examples . . . 29
6.8 Pitfalls . . . 30
6.9 Interview questions . . . 30
6.10 Practice exercises . . . 31

7. Capstone: Live Clock in React . . . 32
7.1 src/utils/time.js — import/export, Date, Math, ternary, destructuring . . . 32
7.2 src/components/Clock.jsx — setInterval + destructuring + ternary . . . 32

7.3 src/App.jsx 33
7.4 src/App.css (minimal) 33
7.5 Stretch goals 34

8. Cheat Sheet 35

SECTION 1

1 import / export (ES Modules)

1.1 The concept

A **module** is just a JavaScript file with its own private scope. Nothing inside it is visible to other files unless you **export** it, and other files cannot use it unless they **import** it.

Before modules, every `<script>` tag shared one global scope, so two files declaring `let user` would collide. Modules fix that: each file is a sealed box with labelled doors.

Real-life analogy: think of a module like a department in an office. The HR team (a file) has lots of internal paperwork (private variables), but it only publishes certain forms (exports). Other departments request those specific forms by name (named imports) or ask for “the main HR form” (default import).

1.2 Named exports

You can have **as many named exports as you want** per file. The importer must use the **exact same name** (inside curly braces).

JavaScript

```
// mathUtils.js
export const PI = 3.14159;
export function add(a, b) {
  return a + b;
}
function subtract(a, b) {
  return a - b;
}
const secret = 42; // not exported → private to this file
export { subtract }; // export an existing binding later in the file
```

JavaScript

```
// app.js
import { add, subtract, PI } from "./mathUtils.js";
console.log(add(2, 3));      // 5
console.log(subtract(9, 4)); // 5
console.log(PI);             // 3.14159
// import { secret } from "./mathUtils.js"; ✗ SyntaxError: no such export
```

1.3 Default export

A file can have **only one** default export. The importer chooses **any name** and uses **no curly braces**.

```JavaScript
// Header.jsx
export default function Header() {
  return <header>ClinicalKey</header>;
}
```

```JavaScript
// App.jsx
import Header from "./Header.jsx";     // ✓
import TopBar from "./Header.jsx";     // ✓ same thing, different local name
```

Things that are **not** allowed:

```JavaScript
export default const x = 5;   // ✗ SyntaxError — default takes an expression, not a
  ↳ declaration
export default 5;             // ✓ fine
const x = 5; export default x; // ✓ fine
```

1.4 Mixing default and named

```JavaScript
// api.js
export const BASE_URL = "https://api.example.com";
export function getUser(id) { /* ... */ }
export default function request(path) { /* ... */ }
```

```JavaScript
import request, { BASE_URL, getUser } from "./api.js"; // default first, then named
```

1.5 Renaming (aliasing) with `as`

```JavaScript
// Avoid name clashes
import { format as formatDate } from "./date.js";
import { format as formatPrice } from "./currency.js";
// Rename while exporting
function internalHelper() {}
export { internalHelper as helper };
```

1.6 Namespace import (`* as`)

Grabs **all named exports** as properties of one object.

```JavaScript
import * as MathUtils from "./mathUtils.js";
MathUtils.add(1, 2);
MathUtils.PI;
MathUtils.default; // the default export (if any) lives here
```

1.7 Re-exports and “barrel” files

A barrel file collects exports from many files so consumers import from one place.

```JavaScript
// components/index.js
export { default as Button } from "./Button.jsx";
export { default as Modal } from "./Modal.jsx";
export * from "./icons.js"; // re-export every named export (NOT the default)
```

```JavaScript
// Anywhere else
import { Button, Modal } from "./components";
```

WATCH OUT

Barrel files are convenient, but very large ones can slow down dev servers and make tree-shaking harder. Use them for a small, stable public API, not for every folder.

1.8 Side-effect imports

Imports that run a file but bind nothing. Very common in React with Vite.

```JavaScript
import "./index.css";        // injects styles
import "./polyfills.js";     // runs setup code
```

1.9 Dynamic `import()` — loading code on demand

Static `import` is at the top of a file and loaded upfront. `import()` is a **function-like expression that returns a Promise**, so you can load code only when needed (code-splitting).

```JavaScript
button.addEventListener("click", async () => {
  const { exportToPdf } = await import("./pdfExporter.js");
  exportToPdf(data);
});
```

In React, this powers lazy-loaded routes:

```JSX
import { lazy, Suspense } from "react";
const Reports = lazy(() => import("./pages/Reports.jsx")); // needs a DEFAULT export
export default function App() {
  return (
    <Suspense fallback={<p>Loading…</p>}>
      <Reports />
    </Suspense>
  );
}
```

Real life: a dashboard where the heavy “Charts” page is only downloaded when the user actually opens it. First page load gets faster.

1.10 How modules behave (the deep part interviewers love)

Behaviour	What it means
Strict mode by default	No accidental globals, `this` at top level is `undefined`.
Imports are hoisted	All `import` statements are processed before any code in the file runs.
Evaluated once (singleton)	Importing the same module from 10 files runs its top-level code only once; everyone shares the same instance.
Live, read-only bindings	If the exporting module changes an exported `let`, importers see the new value. Importers cannot reassign it.
Static structure	`import`/`export` must be at the top level (not inside `if` or functions). This lets bundlers do **tree-shaking** (removing unused exports).
Deferred in the browser	`<script type="module">` is deferred automatically.

Live binding example:

```JavaScript
// counter.js
export let count = 0;
export function increment() { count++; }
```

```JavaScript
// main.js
import { count, increment } from "./counter.js";
console.log(count); // 0
increment();
console.log(count); // 1  ← live binding, not a copy
count = 10;         // ✗ TypeError: Assignment to constant variable
```

Singleton example (real life — a shared API client):

```JavaScript
// apiClient.js
console.log("apiClient created"); // logs ONCE no matter how many files import it
export const apiClient = { token: null };
```

1.11 ES Modules vs CommonJS (full-stack essential)

Node.js historically used **CommonJS**. You will meet both in backend code.

	ES Modules (ESM)	CommonJS (CJS)
Syntax	`import` / `export`	`require()` / `module.exports`
Loading	Static, analysed before running	Dynamic, runs when `require` is called
Bindings	Live	Copy of the value at `require` time
Top-level `await`	✓ Supported	✗ Not supported
Tree-shaking	✓ Easy for bundlers	✗ Hard
Node setup	`"type": "module"` in `package.json` or `.mjs`	Default, or `.cjs`
File path/dir	`import.meta.url`, `import.meta.dirname`	`__filename`, `__dirname`
Extensions in Node	Required: `./utils.js`	Optional

```JavaScript
// CommonJS (older Express code)
const express = require("express");
module.exports = { startServer };
// ESM (modern)
import express from "express";
export { startServer };
```

1.12 `import.meta` in Vite

```JavaScript
const apiUrl = import.meta.env.VITE_API_URL; // env variables in a Vite app
const isDev = import.meta.env.DEV;
```

SECURITY

Only variables prefixed with `VITE_` are exposed to the browser bundle — never put secrets there, because the browser can read them.

1.13 Pitfalls

1. **Curly-brace mismatch:** `import { Header } from "./Header"` when `Header` is a *default* export → `undefined` or a build error. Rule: braces ↔ named, no braces ↔ default.
2. **Circular imports:** `a.js` imports `b.js` which imports `a.js`. One of them may see `undefined` / a TDZ error at load time. Fix by moving shared code into a third file.
3. **Opening `index.html` directly from disk** (`file://`) with modules → CORS errors. Use a dev server (Vite).
4. **Missing `.js` extension in Node ESM** → `ERR_MODULE_NOT_FOUND`.
5. **Anonymous default exports** (`export default () => {}`) show as “Anonymous” in React DevTools and stack traces. Prefer named functions.

1.14 Interview questions

Q1 Named vs default export — when do you use each?

Answer: Named: multiple utilities from one file, enforces consistent names, better for auto-imports and refactoring. Default: one main thing per file (e.g., one React component per file).

Q2 Can a module have multiple default exports?

Answer: No — exactly zero or one.

Q3 What are live bindings?

Answer: Imports are references to the exporter’s variable, not copies. Changes in the exporter are visible to importers.

Q4 What is tree-shaking and why do ES modules enable it?

Answer: Removing unused code at build time. It works because ESM imports/exports are static and analysable before execution.

Q5 Difference between `import` and `import()`?

Answer: Static vs dynamic. `import()` returns a Promise, can be used anywhere, and creates a separate chunk (code-splitting).

Q6 Why does `React.lazy` need a default export?

Answer: It expects the Promise to resolve to a module object with a `default` property. For named exports: `lazy(() => import("./X").then(m => ({ default: m.X })))`.

Q7 How is ESM different from CommonJS?

Answer: See table 1.11.

Q8 What happens if two files import the same module?

Answer: It is evaluated once; both get the same instance.

Q9 Output?

```javascript
// a.js
console.log("A");
import "./b.js";
// b.js
console.log("B");
```

Answer: `B` then `A`, because imports are hoisted and the dependency runs first.

1.15 Practice exercises

☐ 1. Create `utils/string.js` with named exports `capitalize` and `slugify`, plus a default export `truncate`. Import all three in one line.

☐ 2. Create a `components/index.js` barrel that re-exports three components. Import them from `"./components"`.

☐ 3. Build a button that dynamically imports a module and logs “module loaded” only on the first click.

☐ 4. Reproduce the live binding example and confirm the importer sees the new value.

SECTION 2

2 Date

2.1 The concept

A `Date` object stores **one single number**: milliseconds since **1 January 1970, 00:00:00 UTC** (the "Unix epoch"). Everything else — year, month, hour, time zone display — is calculated from that number.

MENTAL MODEL

A Date is a timestamp. Time zones only matter when you *read* it or *write* it as human-friendly text.

2.2 Creating dates

JavaScript

```
new Date();                          // now
new Date(0);                         // 1970-01-01T00:00:00.000Z (epoch)
new Date(1762560000000);             // from a timestamp (ms)
new Date("2026-11-08T10:30:00Z");  // ISO string, UTC ("Z")
new Date("2026-11-08T10:30:00+05:30"); // ISO with offset (IST)
new Date(2026, 10, 8, 10, 30);     // year, MONTH (0-based!), day, hour, min — LOCAL time
Date();          // ⚠ without `new` → returns a STRING, not a Date object
Date.now();      // current timestamp as a number (fastest way)
```

2.3 The famous traps

Trap	Example	Why
Months are 0-indexed	`new Date(2026, 10, 8)` → **November** 8	Jan = 0, Dec = 11. Days of month are 1-based though!
`getDay()` **is the weekday**	`getDay()` → 0 (Sunday) … 6 (Saturday)	Use `getDate()` for day-of-month.
Date-only strings are UTC	`new Date("2026-11-08")` → in India shows **05:30 AM**	ISO date-only = UTC midnight; date-time without zone = local.
Dates are mutable	`setDate()` changes the original object	Copy first: `new Date(d)`.
Overflow rolls over	`new Date(2026, 0, 32)` → 1 Feb 2026	Useful for arithmetic, surprising otherwise.
Invalid dates don't throw	`new Date("hello")` → `Invalid Date`	Check with `isNaN(d.getTime())`.
Non-ISO strings	`new Date("08/11/2026")`	Parsed as **MM/DD** (August 11), and non-ISO formats aren't guaranteed across engines. Always use ISO.

```JavaScript
// Date-only vs date-time — run this in India
new Date("2026-11-08").toString();       // ...Nov 08 2026 05:30:00 GMT+0530
new Date("2026-11-08T00:00").toString(); // ...Nov 08 2026 00:00:00 GMT+0530
```

2.4 Getters (reading parts)

Local time	UTC version	Returns
`getFullYear()`	`getUTCFullYear()`	2026
`getMonth()`	`getUTCMonth()`	0–11
`getDate()`	`getUTCDate()`	1–31
`getDay()`	`getUTCDay()`	0–6 (Sun–Sat)
`getHours()`	`getUTCHours()`	0–23
`getMinutes()` / `getSeconds()` / `getMilliseconds()`	UTC versions too	
`getTime()`	—	ms since epoch (same everywhere)
`getTimezoneOffset()`	—	minutes, **sign reversed**: IST → `-330`

NOTE

✗ Never use `getYear()` — it's deprecated and returns year − 1900.

2.5 Setters (changing parts)

```JavaScript
const d = new Date(2026, 10, 8);
d.setFullYear(2027);
d.setMonth(0);      // January
d.setDate(15);
d.setHours(9, 30, 0, 0); // hours, minutes, seconds, ms in one call
```

Setters **mutate** and **roll over**:

```JavaScript
const jan31 = new Date(2026, 0, 31);
jan31.setMonth(1); // February 31 doesn't exist → becomes 3 March 2026
```

2.6 Formatting for humans

```JavaScript
const d = new Date("2026-11-08T14:05:09+05:30");
d.toISOString();      // "2026-11-08T08:35:09.000Z"  ← always UTC, ideal for APIs/DBs
d.toJSON();           // same as toISOString (used by JSON.stringify)
d.toString();         // full local string
d.toDateString();     // "Sun Nov 08 2026"
d.toLocaleDateString("en-IN");  // e.g. "8/11/2026"
d.toLocaleTimeString("en-IN");  // e.g. "2:05:09 pm"
d.toLocaleDateString("en-US");  // e.g. "11/8/2026"
```

Fine control with options:

```JavaScript
d.toLocaleDateString("en-IN", {
  weekday: "long",
  day: "numeric",
  month: "long",
  year: "numeric",
}); // e.g. "Sunday, 8 November 2026"
d.toLocaleTimeString("en-IN", {
  hour: "2-digit",
  minute: "2-digit",
  hour12: true,
  timeZone: "America/New_York", // show the same instant in another zone
});
```

Reusable and faster when formatting many dates (e.g., a table of 500 rows):

```JavaScript
const fmt = new Intl.DateTimeFormat("en-IN", { dateStyle: "medium", timeStyle: "short"
  ↳ });
rows.map((r) => fmt.format(new Date(r.createdAt)));
```

“3 hours ago” style text:

```JavaScript
const rtf = new Intl.RelativeTimeFormat("en", { numeric: "auto" });
rtf.format(-1, "day");   // "yesterday"
rtf.format(-3, "hour");  // "3 hours ago"
rtf.format(2, "week");   // "in 2 weeks"
```

2.7 Date arithmetic

```JavaScript
const MS_PER_DAY = 24 * 60 * 60 * 1000; // 86_400_000
// Add days (safe across month/year boundaries thanks to rollover)
function addDays(date, days) {
  const copy = new Date(date);       // don't mutate the input
  copy.setDate(copy.getDate() + days);
  return copy;
}
addDays(new Date(2026, 11, 30), 5); // 4 January 2027
// Difference between two dates
const start = new Date("2026-11-01");
const end = new Date("2026-11-08");
const diffDays = Math.round((end - start) / MS_PER_DAY); // 7
// `end - start` works because Dates convert to their timestamp number
// Compare dates
end > start;                              // true
end.getTime() === new Date(end).getTime(); // true
end === new Date(end);                    // false! (different objects)
```

WATCH OUT

Countries with daylight saving have days of 23 or 25 hours, so dividing by `MS_PER_DAY` can be off — that’s why we `Math.round`. India has no DST, but your users in the US/UK do.

2.8 Real-life examples

```JavaScript
// 1. Age calculator (registration form validation)
function getAge(dob, today = new Date()) {
  let age = today.getFullYear() - dob.getFullYear();
  const birthdayPassed =
    today.getMonth() > dob.getMonth() ||
    (today.getMonth() === dob.getMonth() && today.getDate() >= dob.getDate());
  return birthdayPassed ? age : age - 1;
}
// 2. Sprint deadline countdown
function daysLeft(deadline) {
  const ms = new Date(deadline) - Date.now();
  return Math.max(0, Math.ceil(ms / 86_400_000));
}
// 3. Is the date in the past? (e.g., expired subscription)
const isExpired = (expiry) => new Date(expiry).getTime() < Date.now();
// 4. Start of today in local time (for "Today's activity" filters)
const startOfToday = new Date();
startOfToday.setHours(0, 0, 0, 0);
// 5. Validate a date input
function isValidDate(value) {
  const d = new Date(value);
  return !isNaN(d.getTime());
}
```

2.9 Full-stack golden rules

1. **Store in UTC, display in local.** Save `toISOString()` (or a DB `timestamptz`) on the backend; format with `toLocaleString` / `Intl` on the frontend.
2. **Send ISO 8601 strings in APIs** (`"2026-11-08T08:35:09.000Z"`). `JSON.stringify` already does this; on the way back, `JSON.parse` gives you a **string**, so wrap it: `new Date(data.createdAt)`.
3. **Never trust the client clock** for security (token expiry, OTP validity) — check on the server.
4. **Use a library for complex work:** `date-fns` (functional, tree-shakable) or `dayjs` (tiny). The new built-in **Temporal** API is the long-term replacement for `Date` (immutable, time-zone-aware); it has been rolling out in browsers — check current support on MDN/Can I Use before relying on it.

2.10 React connection

```JSX
function PostMeta({ createdAt }) {
  const date = new Date(createdAt);
  return (
    <time dateTime={date.toISOString()}>
      {date.toLocaleDateString("en-IN", { dateStyle: "medium" })}
    </time>
  );
}
```

SSR/HYDRATION

If a server (Next.js) renders `new Date().toLocaleTimeString()` and the browser renders it again a second later, or in a different time zone, React warns about a hydration mismatch. Render time-sensitive text on the client (in `useEffect`) or pass a fixed timestamp.

2.11 Interview questions

Q1 What does a `Date` object actually store?

Answer: A single number: ms since the Unix epoch in UTC.

Q2 Why is `new Date(2026, 1, 1)` in February?

Answer: Months are 0-indexed.

Q3 Difference between `getDate()` and `getDay()`?

Answer: Day of month (1–31) vs day of week (0–6).

Q4 `Date.now()` vs `new Date().getTime()`?

Answer: Same value; `Date.now()` doesn't create an object.

Q5 Why does `new Date("2026-11-08")` show 5:30 AM in India?

Answer: ISO date-only strings are parsed as UTC.

Q6 How do you compare two dates for equality?

Answer: Compare `getTime()` values, not the objects.

Q7 How do you deep-copy a Date?

Answer: `new Date(original)` or `new Date(original.getTime())`. Note `structuredClone` also works; `JSON.parse(JSON.stringify(d))` turns it into a string.

Q8 How would you store dates in a database for a global app?

Answer: UTC timestamps (ISO strings or `timestamptz`), convert at display time.

Q9 Output?

Answer: `typeof Date()` → `"string"`; `typeof new Date()` → `"object"`.

Q10 How do you check for an invalid date?

Answer: `isNaN(date.getTime())` or `Number.isNaN(date.valueOf())`.

2.12 Practice exercises

☐ 1. Write `formatDuration(ms)` that turns `3725000` into `"1h 2m 5s"`.

☐ 2. Write `getDaysInMonth(year, month)`. Hint: `new Date(year, month + 1, 0).getDate()` — day `0` of next month is the last day of this month.

☐ 3. Build `timeAgo(date)` using `Intl.RelativeTimeFormat` that picks seconds/minutes/hours/days automatically.

☐ 4. Print the current time in Chennai, London, and New York using the `timeZone` option.

☐ 5. Write `isWeekend(date)`.

MY NOTES

SECTION 3

3 Math

3.1 The concept

QE CATCH

The slide says `Math()`, but `Math` is **not a function or a constructor**. It's a built-in **namespace object** that holds constants and static methods. `Math()` → `TypeError: Math is not a function`, and `new Math()` fails too. You always write `Math.something(...)`.

3.2 Rounding family — the most-asked section

Method	What it does	4.7	4.3	-4.7	-4.3	-4.5
`Math.round`	nearest integer (.5 goes **up**, towards +∞)	5	4	-5	-4	**-4**
`Math.floor`	always **down** (towards −∞)	4	4	-5	-5	-5
`Math.ceil`	always **up** (towards +∞)	5	5	-4	-4	-4
`Math.trunc`	chops decimals (towards 0)	4	4	-4	-4	-4

TIP

`floor` and `trunc` are identical for positives but differ for negatives. Interviewers love that.

Rounding to N decimal places:

JavaScript

```
Math.round(3.14159 * 100) / 100;   // 3.14  (number)
(3.14159).toFixed(2);              // "3.14" (STRING!)
Number((3.14159).toFixed(2));      // 3.14  (number)
// Floating-point gotcha
0.1 + 0.2;                         // 0.30000000000000004
(1.005).toFixed(2);                // "1.00" not "1.01" — 1.005 is stored as 1.00499999…
Math.round((1.005 + Number.EPSILON) * 100) / 100; // 1.01 ✓ common workaround
```

REAL-WORLD RULE FOR MONEY

Don't do money maths with floating-point rupees. Store **integer paise/cents** (₹199.99 → 19999), calculate with integers, and format only for display with `Intl.NumberFormat("en-IN", { style: "currency", currency: "INR" })`.

3.3 Min, max, absolute, power

```JavaScript
Math.max(3, 9, 2);          // 9
Math.min(3, 9, 2);          // 2
Math.max(...[3, 9, 2]);     // 9  ← spread an array
Math.max([3, 9, 2]);        // NaN ← forgot to spread
Math.max();                 // -Infinity (identity value)
Math.min();                 // Infinity
Math.abs(-7);               // 7
Math.pow(2, 10);            // 1024
2 ** 10;                    // 1024 (modern operator)
Math.sqrt(81);              // 9
Math.cbrt(27);              // 3
Math.sign(-12);             // -1  (0 → 0, positive → 1)
Math.hypot(3, 4);           // 5   (distance √(3² + 4²))
```

WATCH OUT

`Math.max(...hugeArray)` can throw `RangeError` for arrays with hundreds of thousands of items (too many arguments). Use `arr.reduce((a, b) => Math.max(a, b), -Infinity)` instead.

3.4 Constants

```JavaScript
Math.PI;    // 3.141592653589793
Math.E;     // 2.718...
Math.SQRT2; // 1.414...
```

3.5 Randomness

`Math.random()` returns a float in **[0, 1)** — includes 0, never reaches 1.

```JavaScript
// Random integer between min and max (both INCLUSIVE) — memorise this
function randomInt(min, max) {
  return Math.floor(Math.random() * (max - min + 1)) + min;
}
randomInt(1, 6); // dice roll
// Random item from an array
const pick = (arr) => arr[Math.floor(Math.random() * arr.length)];
// Fair shuffle — Fisher–Yates (NOT arr.sort(() => Math.random() - 0.5), which is biased)
function shuffle(arr) {
  const a = [...arr];
  for (let i = a.length - 1; i > 0; i--) {
    const j = Math.floor(Math.random() * (i + 1));
    [a[i], a[j]] = [a[j], a[i]]; // swap via array destructuring
  }
  return a;
}
```

SECURITY

`Math.random()` is **not cryptographically secure**. For OTPs, tokens, passwords or IDs use the Web Crypto API:

```JavaScript
crypto.randomUUID();                                  // "3b1f…" unique ID
crypto.getRandomValues(new Uint32Array(1))[0];    // secure random number
```

3.6 Trigonometry (just enough for UI work)

Math trig functions use **radians**, not degrees.

```JavaScript
const toRad = (deg) => (deg * Math.PI) / 180;
Math.sin(toRad(90)); // 1
Math.cos(toRad(0));  // 1
```

Used in analogue clocks, circular progress rings, charts, and animations. (You'll use this in the capstone.)

3.7 Real-life examples

```JavaScript
// Pagination: 47 test cases, 10 per page → 5 pages
const totalPages = Math.ceil(47 / 10); // 5
// Progress bar percentage (clamped so it never shows 104%)
const clamp = (value, min, max) => Math.min(Math.max(value, min), max);
const percent = clamp(Math.round((passed / total) * 100), 0, 100);
// Average response time, rounded to 1 decimal
const avg = Math.round((times.reduce((a, b) => a + b, 0) / times.length) * 10) / 10;
// Seconds → mm:ss for a video player / OTP timer
const mins = Math.floor(totalSeconds / 60);
const secs = totalSeconds % 60;
// Distance between two points (drag-and-drop threshold)
const moved = Math.hypot(x2 - x1, y2 - y1) > 5;
// Retry with exponential backoff (API calls) — 1s, 2s, 4s, 8s… capped at 30s
const delay = Math.min(1000 * 2 ** attempt, 30_000);
```

3.8 Interview questions

Q1 **Is `Math` a constructor?**

Answer: No, it's a static namespace object.

Q2 **Difference between `Math.floor` and `Math.trunc`?**

Answer: They differ for negative numbers: `floor(-4.3) = -5`, `trunc(-4.3) = -4`.

Q3 **What is `Math.round(-2.5)`?**

Answer: `-2` (halves round toward +∞).

Q4 Why is `0.1 + 0.2 !== 0.3`?

Answer: Numbers are IEEE-754 binary floating point; 0.1 and 0.2 can't be represented exactly. Compare with `Math.abs(a - b) < Number.EPSILON`.

Q5 What does `toFixed` return?

Answer: A string.

Q6 Generate a random integer between 5 and 10 inclusive.

Answer: `Math.floor(Math.random() * 6) + 5`.

Q7 Why not use `Math.random()` for OTPs?

Answer: It's predictable; use `crypto.getRandomValues`.

Q8 Find the max of an array.

Answer: `Math.max(...arr)` (or `reduce` for huge arrays).

Q9 Output?

Answer: `Math.max()` → `-Infinity`; `Math.min()` → `Infinity`; `Math.max(1, "5")` → `5`; `Math.max(1, "a")` → `NaN`.

Q10 Why is `sort(() => Math.random() - 0.5)` a bad shuffle?

Answer: Results are biased and inconsistent across engines; use Fisher–Yates.

3.9 Practice exercises

☐ 1. Write `roundTo(num, decimals)` that returns a number and handles `1.005 → 1.01`.

☐ 2. Write `formatINR(paise)` that turns `1999900` into `"₹19,999.00"` using `Intl.NumberFormat`.

☐ 3. Write a function that generates a random hex colour like `"#a3f1c9"`.

☐ 4. Given an array of test durations, return min, max, and average in one object.

☐ 5. Implement `clamp` and use it to keep a draggable slider value between 0 and 100.

MY NOTES

4

SECTION 4

setInterval()

NOTE

You already went deep on `setInterval` (see your *setInterval_Zero_to_Advanced* notes). This section is the **condensed interview-ready version** plus the React patterns you need for the capstone.

4.1 The concept

`setInterval(callback, delay, ...args)` calls `callback` **repeatedly**, roughly every `delay` milliseconds, until you stop it with `clearInterval(id)`.

```javascript
const id = setInterval(() => console.log("tick"), 1000);
setTimeout(() => clearInterval(id), 5000); // stop after ~5 ticks
```

	`setTimeout`	`setInterval`
Runs	Once	Repeatedly
Stop with	`clearTimeout(id)`	`clearInterval(id)`
Returns	Timer ID (number in browser, `Timeout` object in Node)	Same

4.2 Must-know facts

1. **The delay is a minimum, not a guarantee.** The callback is queued on the event loop; if the main thread is busy, it waits.
2. **It drifts.** Over minutes, a "1000 ms" interval slowly falls behind. For clocks, always read `new Date()` on each tick instead of counting ticks.
3. **Background tabs are throttled** (often to at most once per second, sometimes much less).
4. **Async callbacks can overlap:** if a fetch takes 3s but the interval is 2s, requests pile up. Use recursive `setTimeout` instead:

```javascript
async function poll() {
  await fetchStatus();
  timerId = setTimeout(poll, 2000); // next run starts only after the previous one
    ↳ finishes
}
poll();
```

5. **Always clean up**, or you get memory leaks and "ghost" timers.

4.3 React patterns

Correct live clock:

```jsx
import { useEffect, useState } from "react";
export default function LiveTime() {
  const [now, setNow] = useState(() => new Date());
  useEffect(() => {
    const id = setInterval(() => setNow(new Date()), 1000);
    return () => clearInterval(id); // cleanup on unmount
  }, []);
  return <p>{now.toLocaleTimeString("en-IN")}</p>;
}
```

The stale closure bug (very common interview question):

```jsx
// ✗ count is captured as 0 forever → displays 1, stays 1
useEffect(() => {
  const id = setInterval(() => setCount(count + 1), 1000);
  return () => clearInterval(id);
}, []);
// ✓ functional updater always receives the latest value
useEffect(() => {
  const id = setInterval(() => setCount((c) => c + 1), 1000);
  return () => clearInterval(id);
}, []);
```

StrictMode in development mounts → unmounts → re-mounts components. Without a cleanup function you'll see **two** intervals running and the counter jumps by 2. The cleanup fixes it.

4.4 Real-life examples

- Live clock / dashboard "last updated" label
- OTP "Resend in 30s" countdown
- Auto-save drafts every 30s
- Polling a build/test-run status endpoint
- Image carousel auto-advance (pause on hover with `clearInterval`)

4.5 Interview questions

Q1 Difference between `setTimeout` and `setInterval`?

Answer: Once vs repeated.

Q2 Is the delay exact? Why not?

Answer: No — callbacks run only when the call stack is free (event loop), and browsers throttle/clamp timers.

Q3 Why prefer recursive `setTimeout` for polling?

Answer: Guarantees a gap between runs and avoids overlapping async calls.

Q4 What happens if you forget `clearInterval` in a React component?

Answer: The timer keeps running after unmount → memory leak, state updates on unmounted components, duplicated timers.

Q5 Explain the stale closure problem with `setInterval` in `useEffect`.

Answer: The callback captures state from the first render; fix with functional updates, a ref, or correct dependencies.

Q6 Output order?

```JavaScript
console.log(1);
setTimeout(() => console.log(2), 0);
Promise.resolve().then(() => console.log(3));
console.log(4);
```

Answer: `1, 4, 3, 2` — synchronous code, then microtasks (Promises), then macrotasks (timers).

4.6 Practice exercises

☐ 1. Build a 30-second OTP countdown that disables a “Resend” button until it hits 0.

☐ 2. Build a stopwatch with Start / Pause / Reset buttons (store the interval ID in `useRef`).

☐ 3. Build an auto-rotating carousel that pauses on mouse hover.

MY NOTES

5

SECTION 5

The Ternary Operator

5.1 The concept

The only JavaScript operator with three operands:

```JavaScript
condition ? valueIfTruthy : valueIfFalsy
```

The key insight: a ternary is an **expression** (it produces a value), while `if/else` is a **statement** (it does something). That's exactly why ternaries are everywhere in JSX — **JSX `{ }` only accepts expressions.**

```JavaScript
// if/else (statement)
let label;
if (isLoggedIn) {
  label = "Logout";
} else {
  label = "Login";
}
// ternary (expression) — one line, can be const
const label = isLoggedIn ? "Logout" : "Login";
```

5.2 Truthy / falsy refresher

The condition doesn't need to be a boolean. These are **falsy**: `false`, `0`, `-0`, `0n`, `""`, `null`, `undefined`, `NaN`. Everything else is truthy — including `"0"`, `"false"`, `[]`, and `{}`.

```JavaScript
const items = [];
items ? "has items" : "empty";        // "has items" ([] is truthy)
items.length ? "has items" : "empty"; // "empty" ✓
```

5.3 Real-life examples

```JavaScript
// Pluralisation
const msg = `${count} ${count === 1 ? "test" : "tests"} failed`;
// AM / PM
const period = hours >= 12 ? "PM" : "AM";
// Zero-padding (manual version)
const mm = minutes < 10 ? "0" + minutes : String(minutes);
// …modern alternative: String(minutes).padStart(2, "0")
// Price with discount
const finalPrice = isMember ? price * 0.9 : price;
// API status → badge colour
const badgeColor = status === "passed" ? "green" : "red";
```

5.4 Ternaries in React (the main use)

```jsx
// 1. Swap two pieces of UI
{isLoggedIn ? <Dashboard /> : <LoginForm />}
// 2. Loading / error / data (chained — formatted vertically for readability)
{isLoading ? (
  <Spinner />
) : error ? (
  <ErrorMessage error={error} />
) : (
  <ResultsList results={data} />
)}
// 3. Conditional class names
<button className={isActive ? "tab tab--active" : "tab"}>Search</button>
// 4. Conditional inline style
<p style={{ color: score >= 50 ? "green" : "crimson" }}>{score}</p>
// 5. Render nothing
{showBanner ? <Banner /> : null}
```

5.5 Ternary vs `&&` vs `??` vs `||` vs `?.`

Operator	Use it when	Example
`a ? b : c`	You need **two** outcomes	`{ok ? <Yes/> : <No/>}`
`a && b`	Show something **or nothing**	`{isAdmin && <AdminPanel/>}`
`a || b`	Fallback when a is **any falsy** value	`name || "Guest"`
`a ?? b`	Fallback only when a is `null`/`undefined`	`count ?? 0` (keeps a real `0`)
`a?.b`	Safely access a property that may not exist	`user?.address?.city`

The `&&` zero bug (classic React interview question):

```jsx
const count = 0;
{count && <p>{count} new messages</p>}     // ✗ renders "0" on screen!
{count > 0 && <p>{count} new messages</p>} // ✓
{count ? <p>{count} new messages</p> : null} // ✓
```

React doesn't render `false`, `null`, or `undefined`, but it **does** render the number `0`.

5.6 Pitfalls and style

1. **Don't nest deeply.** More than two levels is hard to read. Use early returns, a lookup object, or a helper function:

```jsx
// ✗ hard to read
const icon = s === "pass" ? "✓" : s === "fail" ? "✗" : s === "skip" ? "»" : "?";
// ✓ lookup object
const ICONS = { pass: "✓", fail: "✗", skip: "»" };
const icon = ICONS[s] ?? "?";
```

2. **Don't use ternaries for side effects.** `isValid ? submit() : showError()` works, but `if/else` is clearer for actions.

3. **Avoid redundant ternaries:** `isDone ? true : false` → just `isDone` (or `Boolean(x)` / `!!x`).
4. **Associativity:** ternaries are right-associative, so `a ? b : c ? d : e` means `a ? b : (c ? d : e)`.

5.7 Interview questions

Q1 Why can't you use `if/else` inside JSX braces?

Answer: JSX `{ }` accepts expressions only; `if` is a statement. Use a ternary, `&&`, or compute the value before `return`.

Q2 Difference between `||` and `??`?

Answer: `||` falls back on any falsy value (including `0` and `""`); `??` only on `null`/`undefined`.

Q3 Why does `{count && <X/>}` show `0`?

Answer: `&&` returns the first falsy operand (`0`), and React renders numbers.

Q4 Is a ternary faster than `if/else`?

Answer: No meaningful difference; choose for readability.

Q5 Output?

Answer: `const x = "" ? "A" : "0" ? "B" : "C";` → `"B"` (`""` is falsy, `"0"` is truthy).

Q6 How do you render nothing with a ternary?

Answer: Return `null` in the false branch.

5.8 Practice exercises

☐ 1. Write `getGrade(score)` (A/B/C/F) first with nested ternaries, then refactor to readable code.

☐ 2. Build a React `<StatusBadge status="passed" />` that changes text and colour with a ternary.

☐ 3. Build a "Show more / Show less" toggle where both the button text and the content length depend on a ternary.

☐ 4. Fix the `&&` zero bug in a notifications counter.

SECTION 6

6 Object Destructuring

6.1 The concept

Destructuring **unpacks values from an object (or array) into separate variables** in one line. It doesn't change the original object — it just reads from it.

```JavaScript
const user = { name: "Selva", role: "QE", city: "Chennai" };
// Without destructuring
const name = user.name;
const role = user.role;
// With destructuring — matches by PROPERTY NAME, order doesn't matter
const { role, name } = user;
```

Real-life analogy: unpacking a parcel. Instead of saying "parcel.charger, parcel.cable", you take out the charger and the cable and put them on the table by name.

6.2 Every feature, one by one

```JavaScript
const response = {
  id: 101,
  title: "Login button misaligned",
  status: "open",
  assignee: { name: "Selva", team: "QA" },
  tags: ["ui", "header"],
  priority: null,
};
// 1. Rename (alias)  — source : newName
const { id: ticketId } = response;          // ticketId = 101
// 2. Default value  — used ONLY when the value is undefined
const { severity = "medium" } = response;   // "medium"
const { priority = "low" } = response;      // null  ⚠ (null is not undefined)
// 3. Rename + default together
const { severity: sev = "medium" } = response;
// 4. Nested
const { assignee: { name, team } } = response; // name="Selva", team="QA"
// Note: `assignee` itself is NOT a variable here — only name and team are.
// 5. Rest — collect remaining properties into a new object
const { id, title, ...rest } = response;
// rest = { status, assignee, tags, priority }
// 6. Computed property name
const field = "status";
const { [field]: currentStatus } = response; // "open"
// 7. Mixed object + array
const { tags: [firstTag] } = response;       // "ui"
```

6.3 Destructuring in function parameters (the React superpower)

```JavaScript
// Without
function greet(user) {
  return `Hi ${user.name} from ${user.city}`;
}
// With destructuring + defaults
function greet({ name, city = "Unknown" }) {
  return `Hi ${name} from ${city}`;
}
// Make the whole argument optional — default to {}
function createTest({ title = "Untitled", priority = "Medium" } = {}) {
  return { title, priority };
}
createTest(); // ✓ without "= {}" this would throw
```

“Options object” pattern (used by `fetch`, Playwright, Axios, etc.): named arguments, any order, easy defaults.

```JavaScript
function fetchTests({ project, status = "open", page = 1, pageSize = 20 }) { /* ... */ }
fetchTests({ project: "GAA", page: 2 });
```

6.4 Assigning to existing variables

```JavaScript
let a, b;
{ a, b } = { a: 1, b: 2 };   // ✗ SyntaxError — `{` starts a block
({ a, b } = { a: 1, b: 2 }); // ✓ wrap in parentheses
```

6.5 Array destructuring (you use it daily in React)

Arrays destructure by **position**, not by name.

```JavaScript
const [first, second] = ["red", "green", "blue"];  // "red", "green"
const [, , third] = ["red", "green", "blue"];      // skip with commas → "blue"
const [head, ...tail] = [1, 2, 3, 4];              // 1, [2, 3, 4]
const [x = 10] = [];                               // default → 10
// Swap without a temp variable
let p = 1, q = 2;
[p, q] = [q, p];
// useState is array destructuring!
const [count, setCount] = useState(0);
// Why array and not object? So YOU choose the names: [isOpen, setIsOpen], [user,
  ↳ setUser]…
```

6.6 Destructuring in loops

```JavaScript
const scores = { Selva: 92, Rory: 88 };
for (const [name, score] of Object.entries(scores)) {
  console.log(`${name}: ${score}`);
}
const users = [{ id: 1, name: "A" }, { id: 2, name: "B" }];
users.map(({ id, name }) => `${id}-${name}`);
```

6.7 Real-life examples

Frontend — API response:

```JavaScript
const res = await fetch("/api/tests?page=1");
const { data, meta: { total, page } } = await res.json();
```

Frontend — React props:

```JSX
// Without destructuring
function TestCard(props) {
  return <h3>{props.title} — {props.status}</h3>;
}
// With destructuring + defaults + rest-spread
function TestCard({ title, status = "draft", onSelect, ...otherProps }) {
  return (
    <article {...otherProps} onClick={onSelect}>
      <h3>{title}</h3>
      <span>{status === "passed" ? "✓" : "…"}</span>
    </article>
  );
}
```

Backend — Express route:

```JavaScript
app.post("/api/users/:id/tests", (req, res) => {
  const { id } = req.params;
  const { title, steps = [] } = req.body;
  const { page = "1" } = req.query;
  // ...
});
```

Remove a field immutably (e.g., hide a password before sending a user to the client):

```JavaScript
const { password, ...safeUser } = userFromDb;
res.json(safeUser);
```

Returning multiple values from a function:

```JavaScript
function getStats(arr) {
  return { min: Math.min(...arr), max: Math.max(...arr) };
}
const { min, max } = getStats([4, 9, 1]);
```

6.8 Pitfalls

1. **Destructuring** `null`/`undefined` **throws:**

   ```JavaScript
   const { name } = undefined; // ✗ TypeError: Cannot destructure property 'name'
   const { name } = data ?? {}; // ✓ safe fallback
   ```

   Very common when API data hasn't loaded yet in React.

2. **Defaults ignore** `null` — only `undefined` triggers them.
3. **Nested destructuring crashes on a missing parent:** `const { assignee: { name } } = {}` → TypeError. Add a default: `const { assignee: { name } = {} } = obj`.
4. **Rest makes a *shallow* copy:** nested objects are still shared references.
5. **Over-destructuring** deep structures in one line hurts readability. Destructure in steps.

6.9 Interview questions

Q1 What is destructuring?

Answer: Syntax to unpack object properties (by name) or array items (by position) into variables.

Q2 How do you rename while destructuring?

Answer: `const { oldName: newName } = obj`.

Q3 When does a default value apply?

Answer: Only when the value is `undefined` — not `null`.

Q4 Why does `useState` return an array instead of an object?

Answer: So the caller can name the pair freely without aliasing syntax.

Q5 How do you remove a property without mutating the object?

Answer: `const { unwanted, ...rest } = obj`.

Q6 Why the parentheses in `({ a } = obj)`?

Answer: Without them, `{` at the start of a statement is parsed as a block.

Q7 Is the rest object a deep copy?

Answer: No, shallow.

Q8 Output?

```JavaScript
const { a = 1, b = 2 } = { a: null, b: undefined };
console.log(a, b);
```

Answer: `null 2`.

Q9 How do you safely destructure props that might be missing?

Answer: Defaults in the parameter list and a fallback `= {}` for the whole object.

6.10 Practice exercises

☐ 1. Given a nested API response, extract `user.profile.email` and rename it to `userEmail` with a default of `"N/A"`.

☐ 2. Write `omit(obj, key)` using computed keys + rest: `const { [key]: _, ...rest } = obj`.

☐ 3. Refactor a React component that uses `props.x` everywhere to destructured props with defaults.

☐ 4. Swap two variables using array destructuring.

☐ 5. Loop over `Object.entries` of a settings object and print `key = value`.

MY NOTES

SECTION 7

7 Capstone: Live Clock in React

This mini-project uses **all six topics**. Create a Vite React app and add these files.

7.1 src/utils/time.js — import/export, Date, Math, ternary, destructuring

JavaScript

```
// Named export — tiny helper
export const pad = (n) => String(n).padStart(2, "0");
// Named export — break a Date into display parts
export function getClockParts(date) {
  const h24 = date.getHours();
  return {
    hours: pad(h24 % 12 || 12),       // 0 → 12 (midnight), 13 → 1
    minutes: pad(date.getMinutes()),
    seconds: pad(date.getSeconds()),
    period: h24 >= 12 ? "PM" : "AM",  // ternary
  };
}
// Named export — angles for analogue hands (Math)
export function getHandAngles(date) {
  const s = date.getSeconds();
  const m = date.getMinutes();
  const h = date.getHours() % 12;
  return {
    secondDeg: s * 6,                 // 360° / 60 s
    minuteDeg: m * 6 + s * 0.1,       // + smooth movement from seconds
    hourDeg: h * 30 + m * 0.5,        // 360° / 12 h, + 0.5° per minute
  };
}
// Default export — greeting by time of day (chained ternary, still readable)
export default function getGreeting(hour) {
  return hour < 12 ? "Good morning" : hour < 17 ? "Good afternoon" : "Good evening";
}
```

TIP

`h24 % 12 || 12` — `%` gives 0 for 12 AM/PM, and since `0` is falsy, `|| 12` replaces it with 12. One-line 24h → 12h conversion.

7.2 src/components/Clock.jsx — setInterval + destructuring + ternary

JSX

```
import { useEffect, useState } from "react";
import getGreeting, { getClockParts, getHandAngles } from "../utils/time.js";
export default function Clock({ name = "there", showSeconds = true }) {
  const [now, setNow] = useState(() => new Date());
  useEffect(() => {
    const id = setInterval(() => setNow(new Date()), 1000); // read real time every tick
    return () => clearInterval(id);
```

```
  }, []);
  const { hours, minutes, seconds, period } = getClockParts(now);
  const { hourDeg, minuteDeg, secondDeg } = getHandAngles(now);
  const dateText = now.toLocaleDateString("en-IN", {
    weekday: "long",
    day: "numeric",
    month: "long",
    year: "numeric",
  });
  return (
    <section className="clock">
      <h2>{getGreeting(now.getHours())}, {name}!</h2>
      <div className="dial">
        <span className="hand hour" style={{ transform: `rotate(${hourDeg}deg)` }} />
        <span className="hand minute" style={{ transform: `rotate(${minuteDeg}deg)` }} />
        {showSeconds ? (
          <span className="hand second" style={{ transform: `rotate(${secondDeg}deg)` }}
            ↳ />
        ) : null}
      </div>
      <p className="digital">
        {hours}:{minutes}
        {showSeconds ? `:${seconds}` : ""} {period}
      </p>
      <time dateTime={now.toISOString()}>{dateText}</time>
    </section>
  );
}
```

7.3 src/App.jsx

```jsx
import Clock from "./components/Clock.jsx";
import "./App.css"; // side-effect import
export default function App() {
  return <Clock name="Selva" showSeconds />;
}
```

7.4 src/App.css (minimal)

```css
.clock { display: grid; place-items: center; gap: 1rem; font-family: Inter, system-ui,
  ↳ sans-serif; }
.dial { position: relative; width: 12rem; height: 12rem; border: 4px solid currentColor;
  ↳ border-radius: 50%; }
.hand { position: absolute; bottom: 50%; left: 50%; transform-origin: bottom center;
  ↳ background: currentColor; border-radius: 4px; }
.hour { width: 6px; height: 28%; margin-left: -3px; }
.minute { width: 4px; height: 40%; margin-left: -2px; }
.second { width: 2px; height: 45%; margin-left: -1px; background: crimson; }
.digital { font-size: 2rem; font-variant-numeric: tabular-nums; }
```

WATCH OUT

Because each `span` sets `transform` inline, the `translateX(-50%)` trick would be overwritten — that's why centring uses a negative `margin-left` instead. Good thing to notice when debugging!

7.5 Stretch goals

☐ 1. Add a `timeZone` prop and show three world clocks (Chennai, London, New York) side by side.

☐ 2. Add a Pomodoro countdown (25:00) with Start / Pause / Reset using `useRef` for the interval ID.

☐ 3. Lazy-load a “Stopwatch” tab with `React.lazy(() => import(...))`.

☐ 4. Write a unit test (Vitest) for `getClockParts` with fixed dates: midnight, noon, 23:59:59. Edge cases are your QE strength — use them!

MY NOTES

SECTION 8

8 Cheat Sheet

Topic	Remember this
import/export	Braces ↔ named, no braces ↔ default. One default per file. Imports are hoisted, live, read-only, evaluated once. `import()` returns a Promise (code-splitting). ESM vs CommonJS.
Date	Stores ms since epoch (UTC). Months 0–11, `getDay()` = weekday. Date-only ISO = UTC. Mutable — copy before `set*`. Store UTC, display local with `Intl`. `isNaN(d.getTime())` for invalid.
Math	Namespace object, not a function. `round/floor/ceil/trunc` differ on negatives. `toFixed` → string. Random int: `floor(random() * (max-min+1)) + min`. Use `crypto` for security. Money in integer paise.
setInterval	Delay is a minimum. Drifts → read `new Date()` each tick. Always `clearInterval` (React cleanup). Functional updater to avoid stale closures. Recursive `setTimeout` for async polling.
Ternary	Expression, so it works inside JSX. `?:` two outcomes, && one-or-nothing (beware `0`), ?? null/undefined fallback. Avoid deep nesting.
Destructuring	Objects by name, arrays by position. `a: b` renames, `= x` defaults (only for `undefined`). `...rest` = shallow copy without some keys. Wrap assignment in `()`. Destructuring `null` throws → `?? {}`.

Next steps: do the exercises in each section, build the capstone, then try the stretch goals. Bring your solutions back and I'll review them like a code review.